



Intelligence Academy

Scikit-Learn 101

09. Fit for supervised models With Python

Mejbah Ahammad

I. The Supervised Learning Paradigm

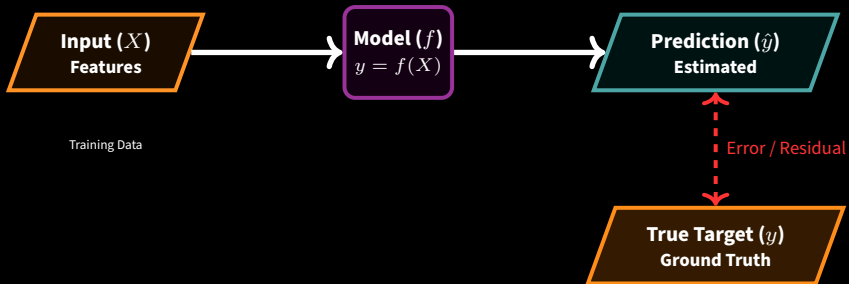
Goal: Mapping Inputs (X) to Targets (y)

Core Concept: Supervised learning is the process of learning a function that maps an input to an output based on example input-output pairs.

$$\hat{y} = f(X; \theta) \approx y$$

- **Features (X):** The input variables (matrix of shape $N \times D$).
- **Targets (y):** The "ground truth" labels we want to predict.

The objective is to find the parameters θ such that the predictions $\hat{y}$ are as close as possible to the true targets y for unseen data.



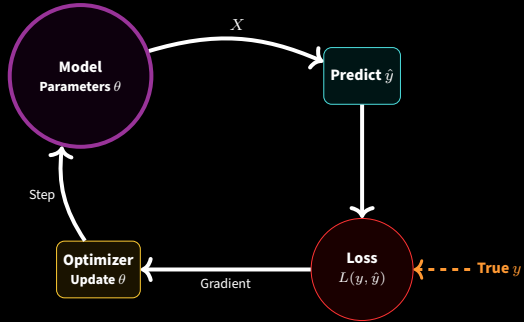
The Role of the Loss Function in Training

How do we measure "success"?

- **The Metric:** A **Loss Function** $L(y, \hat{y})$ quantifies the penalty for bad predictions.
- **The Goal:** Training is an optimization problem where we minimize this loss.

$$\min_{\theta} \sum_{i=1}^N L(y^{(i)}, f(X^{(i)}; \theta))$$

The `.fit()` method runs an iterative loop: **Predict** → **Calculate Loss** → **Update Model**.



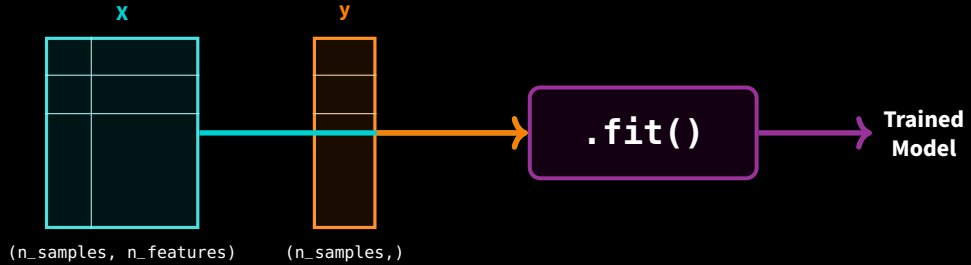
II. The `.fit(X, y)` Method

Standard Syntax and Arguments

In Scikit-Learn, the API is consistent across all supervised models. The training process is always triggered by the `fit` method.

`model.fit(X, y)`

- **X (Features):** A 2D array-like structure (Matrix).
Shape: `(n_samples, n_features)`
- **y (Target):** A 1D array-like structure (Vector).
Shape: `(n_samples,)`
- **Returns:** The estimator object itself (allows method chaining).



Statefulness: How a Model Changes

Before vs. After: A Scikit-Learn model is "stateful." Calling `.fit()` mutates the object by adding new attributes.

1. Initialization (Constructor):

Only stores **Hyperparameters** (configuration).

Example: `LinearRegression(fit_intercept=True)`

2. After Fitting:

Calculates and stores **Learned Parameters**.

Naming Convention: Learned attributes always end with an **underscore** (`_`).

Example: `model.coef_`, `model.intercept_`

```
// State 1: Unfitted  
LinearRegressionObject
```

```
-----
```

```
fit_intercept = True
```

```
n_jobs = None
```

```
coef_ <Not Exists>
```

```
intercept_ <Not Exists>
```

.fit(X, y)

```
// State 2: Fitted
```

```
LinearRegressionObject
```

```
-----
```

```
fit_intercept = True
```

```
n_jobs = None
```

```
coef_ = [0.45, -1.2]
```

```
intercept_ = 3.14
```

IV. Python Implementation

Step-by-Step Code Walkthrough

The Universal Pattern: Almost every Scikit-Learn model follows this exact 3-step recipe. Memorizing this unlocks 90% of the library.

1. **Import:** Bring in the specific class you need.
2. **Instantiate:** Create an object and set **Hyperparameters**.
3. **Fit:** Feed the data (X, y) to learn **Parameters**.

```
1  # 1. Import the class
2  from sklearn.linear_model import LinearRegression
3
4  # 2. Instantiate the model (Hyperparameters)
5  model = LinearRegression(fit_intercept=True)
6
7  # 3. Fit the model to data (Learning)
8  # X_train must be 2D, y_train 1D
9  model.fit(X_train, y_train)
```

The model object is now trained and holds the state.

Case Study: Training LinearRegression

Let's train a model on simple dummy data to verify the shapes and attributes.

Goal: Learn the relationship $y = 2x$.

- X : $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ (Shape 3×1)
- y : $[2, 4, 6]$ (Shape 3)

```
1  import numpy as np
2  from sklearn.linear_model import LinearRegression
3
4  # Create dummy data (Note X is 2D)
5  X = np.array([[1], [2], [3]])
6  y = np.array([2, 4, 6])
7
8  # Train
9  regr = LinearRegression()
10 regr.fit(X, y)
11
12 # Check learned attributes (suffixed with _)
13 print(f"Slope: {regr.coef_}")      # Output: [2.]
14 print(f"Bias: {regr.intercept_}") # Output: 0.0
```

✓ **Training Successful**
coef_ learned accurately

Logic:

$$\text{Input } x = 1 \rightarrow 2(1) + 0 = 2$$

$$\text{Input } x = 2 \rightarrow 2(2) + 0 = 4$$

V. Data Shape & Dimensionality

Critical Constraints: The Shape Rules

Scikit-Learn is strict about data dimensions. The `.fit()` method assumes specific mathematical structures for efficient linear algebra operations.

- **Feature Matrix (X): MUST be 2D.**

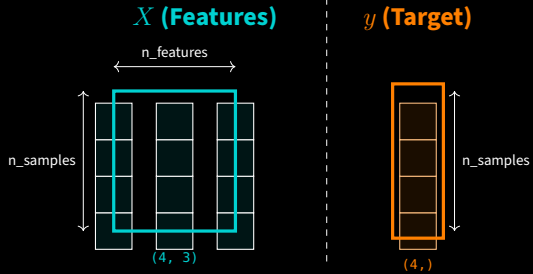
Shape: `(n_samples, n_features)`

Even if you only have **one feature**, it must be a column vector, not a flat list.

- **Target Vector (y): Usually 1D.**

Shape: `(n_samples,)`

A simple flat array of labels or values.



Common Error Handling: Reshaping

The Error: If you pass a 1D array (e.g., [1, 2, 3]) as X , you will see:

ValueError: Expected 2D array, got 1D array instead.

The Fix: Reshape your data to add a second dimension.

```
1 import numpy as np
2 x_bad = np.array([1, 2, 3, 4]) # Shape (4,) -> Error!
3
4 # Reshape to (n_samples, 1)
5 # -1 means "calculate this dimension automatically"
6 X_good = x_bad.reshape(-1, 1) # Shape (4, 1) -> OK!
```

